

Here is the software manual for your Godot Rithmomachia package, based on the board.gd and move_validator.gd scripts provided.

Rithmomachia for Godot: Software Manual

1. Introduction

What is Rithmomachia?

Rithmomachia, also known as "The Philosopher's Game," is a complex mathematical board game from medieval Europe. It was used as an educational tool to teach arithmetic, proportion, and mathematical theory. The game involves two armies, White and Black, with pieces numbered according to specific mathematical progressions. Pieces move orthogonally based on their shape (Circle, Triangle, Square), and captures are made not by landing on a piece, but by using mathematical relationships (equality, sum, difference, product, division) between the attacking piece(s) and the target.

Package Overview

This package provides a comprehensive implementation of Rithmomachia for the Godot game engine. It includes all necessary logic for board setup, piece movement, capture validation, turn management, AI, and game state tracking.

The logic is primarily split into two main scripts:

- **board.gd**: The main game controller. It manages the board state, player turns, piece nodes, UI, game logging, and executes game actions like moving and capturing. It acts as

the "server" for the game.

- **move_validator.gd:** The rules engine. This script contains pure, stateless functions that validate moves and identify all possible captures based on the game's complex mathematical rules. It is instantiated by board.gd.

This manual details the main callable functions (the public API) of these scripts, organized by topic.

2. Game State & Turn Management

These functions, found in **board.gd**, control the flow of the game, manage turn phases, and handle saving/loading the game state.

advance_to_move_phase()

Advances the game from the PRE_MOVE_CAPTURE phase to the MOVE phase. This typically happens after the player presses the "Done Capturing" button.

- **Syntax Example:**

```
GDScript
# Called by the phase button's 'pressed' signal
board.advance_to_move_phase()
```

advance_to_next_player()

Finalizes the current player's turn (logs the turn, updates scores) and switches control to the other player, resetting the phase to PRE_MOVE_CAPTURE.

- **Syntax Example:**

```
GDScript
# Called by the "End Turn" button
board.advance_to_next_player()
```

advance_to_post_capture_phase()

Advances the game from the MOVE phase to the POST_MOVE_CAPTURE phase. This is called automatically after a successful move.

- **Syntax Example:**

```
GDScript
# Called internally after 'execute_move'
board.advance_to_post_capture_phase()
```

capture_board_state()

Creates and returns a dictionary containing a snapshot of the entire game state. This includes all piece positions, pyramid values, and a deep copy of the initial_board_state array. Used for the undo system.

- **Syntax Example:**

```
GDScript
var current_state = board.capture_board_state()
var piece_id = current_state.piece_positions["0,0"].id
```

check_if_player_can_move()

Checks if the current_player has any legal moves available on the board. If not, it automatically calls show_game_over() to end the game.

- **Syntax Example:**

```
GDScript
if not board.check_if_player_can_move():
    print("Game over, player is stalemated.")
```

`declare_victory()`

Ends the game and declares a winner based on the victory type. This logs the final turn, saves the game log, and updates the UI.

- **Syntax Example:**

```
GDScript
# Called internally by on_piece_captured
board.declare_victory("White", "Goods")
```

`restore_board_state()`

Restores the game to a previously captured state. It clears the board and capture zones, then repopulates them based on the provided state dictionary.

- **Syntax Example:**

```
GDScript
var old_state = game_log.pop_back().board_state_snapshot
board.restore_board_state(old_state)
```

`show_game_over()`

Displays the game over message on the UI and disables the main phase button.

- **Syntax Example:**

```
GDScript
board.show_game_over("Black wins! White has no legal moves.")
```

3. Game Actions

These functions, found in **board.gd**, execute a direct change to the game state, such as moving or capturing a piece.

`attempt_capture()`

The primary controller for captures. When given an attacker and victim tile, it queries `move_validator` for all valid capture types and executes the first one found. It handles logging and calls the appropriate piece/subpiece removal function.

- **Syntax Example:**

```
GDScript
var attacker_tile = board.get_tile_at_coords(2, 2)
var victim_tile = board.get_tile_at_coords(2, 4)
board.attempt_capture(attacker_tile, victim_tile)
```

`attempt_move()`

The primary controller for movement. When given a `from_tile` and `to_tile`, it checks with `move_validator` if the move is valid. If so, it calls `execute_move()` and advances the game phase.

- **Syntax Example:**

```
GDScript
# Assumes 'selected_tile' and 'clicked_tile' are set
board.attempt_move(selected_tile, clicked_tile)
```

`execute_full_piece_removal()`

Removes an entire piece (non-pyramid) from the board, updates the internal board state, and moves the piece's node to the captured pieces container.

- **Syntax Example:**

```
GDScript
var victim_tile = board.get_tile_at_coords(5, 5)
board.execute_full_piece_removal(victim_tile)
```

`execute_move()`

Performs the physical act of moving a piece node from one tile to another and updates the `initial_board_state` array. It also logs the move.

- **Syntax Example:**

```
GDScript
var from_tile = board.get_tile_at_coords(1, 1)
var to_tile = board.get_tile_at_coords(1, 4)
board.execute_move(from_tile, to_tile)
```

`execute_pyramid_subpiece_removal()`

Handles the capture of a pyramid subpiece. It removes the `captured_value` from the pyramid's internal label, creates a new display piece for the capture zone, and removes the full pyramid node if its label becomes empty.

- **Syntax Example:**

```
GDScript
var pyramid_piece_node = get_node("PieceLayer/P091_1")
var pyramid_pos = Vector2i(1, 1)
board.execute_pyramid_subpiece_removal(pyramid_piece_node, 16, pyramid_pos)
```

`undo_last_turn()`

Pops the last turn from the `game_log`, retrieves its `board_state_snapshot`, and calls `restore_board_state()` to revert the game. It also restores turn/phase data and captured piece lists.

- **Syntax Example:**

```
GDScript
# Called by the Undo Button
board.undo_last_turn()
```

4. Board & Piece Queries

These functions get information about the game state without changing it. They are sourced from both **board.gd** and **move_validator.gd**.

count_all_progressions()

A wrapper function that returns a dictionary containing progression counts (arithmetical, geometric, harmonic) for both "on board" and "invader pairs" for both players.

- **Syntax Example:**

```
GDScript
var progressions = board.count_all_progressions()
var white_invaders = progressions.white_invaders.total
```

count_invader_pairs_for_color()

Counts how many pairs of pieces from valid 3-piece progressions are currently in enemy territory. This is used as a component for the AI's position score.

- **Syntax Example:**

```
GDScript
var white_invader_data =
board.count_invader_pairs_for_color("white")
print("White invader pairs: ", white_invader_data.total)
```

count_progressions_for_color()

Counts how many complete 3-piece progressions (arithmetical, geometric, harmonic) a given color has anywhere on the board.

- **Syntax Example:**

```
GDScript
var black_progressions = board.count_progressions_for_color("black")
print("Black has ", black_progressions.geometric, " geometric progressions.")
```

`find_progression_on_board()`

Searches the board for all instances of a specific 3-value pattern (e.g., [2, 4, 6]) for a given color. Returns an array of results.

- **Syntax Example:**

```
GDScript
var pattern = [2, 4, 8] # White geometric
var found = board.find_progression_on_board(pattern, "white")
if found.size() > 0:
    print("White has a 2-4-8 progression on the board.")
```

`get_all_capturable_enemy_pieces()`

Scans the entire board and returns a sorted array of all enemy pieces that the attacking_color can currently capture. Game-winning captures are sorted to the top.

- **Syntax Example:**

```
GDScript
var targets = board.get_all_capturable_enemy_pieces("white")
if targets.size() > 0:
    print("White can capture: ", targets[0].piece.piece_id)
```

`get_all_piece_positions_with_values()`

Returns an array of dictionaries, one for each piece of the specified color, containing its position and value (total value for pyramids).

- **Syntax Example:**

```
GDScript
var white_pieces =
board.get_all_piece_positions_with_values("white")
for p in white_pieces:
    print(p.piece.piece_id, " at ", p.pos, " has value ",
p.value)
```


`get_all_possible_captures()`

Returns an array of all possible captures for a single `captor_piece`. Each entry in the array is a dictionary detailing the target, the value used, and the type(s) of capture.

- **Syntax Example:**

```
GDScript
var piece_node = board.get_tile_at_coords(0, 0).get_child(0)
var pos = Vector2i(0, 0)
var captures = move_validator.get_all_possible_captures(pos, piece_node)
```

`get_all_possible_moves()`

Returns an array of all valid moves (`{"from": Vector2i, "to": Vector2i}`) for all pieces of a given color.

- **Syntax Example:**

```
GDScript
var all_white_moves =
move_validator.get_all_possible_moves("white")
print("White has ", all_white_moves.size(), " legal moves.")
```

`get_hanging_pieces_summary()`

Returns a dictionary summarizing the "material advantage" for a given color, detailing which friendly pieces are at risk and which enemy pieces can be captured.

- **Syntax Example:**

```
GDScript
var summary = board.get_hanging_pieces_summary("black")
print("Black's material advantage: ", summary.material_advantage)
```

`get_path_clear_and_distance()`

A helper function that checks if the orthogonal path between two positions is empty. If it is, the function returns a dictionary {"clear": true, "distance": int}, where distance is the number of empty squares *between* them.

- **Syntax Example:**

```
GDScript
var start = Vector2i(1, 1)
var end = Vector2i(1, 5)
var path_data = move_validator.get_path_clear_and_distance(start, end)
if path_data.clear:
    print("Path is clear, distance is: ", path_data.distance) # Output: 3
```

`get_piece_value()`

A helper function that returns the total numerical value of a piece node (sums subpieces for Pyramids).

- **Syntax Example:**

```
GDScript
var piece_node = board.get_tile_at_coords(0, 0).get_child(0)
var value = board.get_piece_value(piece_node)
```

`get_pieces_in_enemy_territory()`

Returns an array of all piece nodes for a given color that are currently on the opponent's side of the board.

- **Syntax Example:**

```
GDScript
var white_invaders = board.get_pieces_in_enemy_territory("white")
print("White has ", white_invaders.size(), " pieces in enemy territory.")
```

`get_potential_moves()`

Returns an array of Vector2i positions a piece *could* move to based on its shape (Circle=1, Triangle=2, Square=3), *ignoring* any blocking pieces.

- **Syntax Example:**

```
GDScript
var piece_node = board.get_tile_at_coords(1, 1).get_child(0)
var pos = Vector2i(1, 1)
var moves = move_validator.get_potential_moves(piece_node, pos)
```

`get_tile_at_coords()`

A utility function to get the ColorRect (tile) node at a specific (x, y) coordinate. Returns null if out of bounds.

- **Syntax Example:**

```
GDScript
var tile = board.get_tile_at_coords(0, 0)
```

`get_tile_coords()`

A utility function to get the Vector2i(x, y) coordinates of a given ColorRect (tile) node.

- **Syntax Example:**

```
GDScript
var tile_node = $MainLayout/BoardAndColumnLabels/GridContainer.get_child(0)
var coords = board.get_tile_coords(tile_node) # Returns Vector2i(0, 0)
```

`get_valid_moves()`

Returns an array of `Vector2i` positions a piece can legally move to. This checks for board bounds, path obstructions, and that the destination is empty.

- **Syntax Example:**

```
GDScript
var piece_node = board.get_tile_at_coords(0, 2).get_child(0)
var pos = Vector2i(0, 2)
var valid_moves = move_validator.get_valid_moves(piece_node, pos)
```

`get_winning_captures()`

A convenience function that filters the output of `get_all_capturable_enemy_pieces()` to return only those captures that would win the game.

- **Syntax Example:**

```
GDScript
var winning_moves = board.get_winning_captures("white")
if winning_moves.size() > 0:
    print("White has a winning capture!")
```

`has_winning_capture_available()`

A quick boolean check to see if the `attacking_color` has any game-winning captures available.

- **Syntax Example:**

```
GDScript
if board.has_winning_capture_available("black"):
    # AI should prioritize this
    pass
```

5. Move & Capture Validation

These functions return a boolean or simple value to validate a specific game rule.

`check_if_values_form_progression()`

Checks if three integer values (v1, v2, v3) form one of the valid progressions (arithmetical, geometric, harmonic) for the specified color.

- **Syntax Example:**

```
GDScript
var result = board.check_if_values_form_progression(2, 4, 8,
"white")
if result.is_valid:
    print(result.type) # Output: "geometric"
```

`check_victory_by_body()`

Checks if the captured_count meets or exceeds the victory_n0 threshold.

- **Syntax Example:**

```
GDScript
if board.check_victory_by_body("White", 10):
    board.declare_victory("White", "Body")
```

`check_victory_by_goods()`

Checks if the captured_value meets or exceeds the victory_n1 threshold.

- **Syntax Example:**

```
GDScript
if board.check_victory_by_goods("Black", 401):
    board.declare_victory("Black", "Goods")
```

is_blockaded()

Checks if a piece at a given pos has any valid moves. Returns true if it has zero valid moves (is "blockaded" or "besieged").

- **Syntax Example:**

```
GDScript
var pos = Vector2i(3, 3)
if move_validator.is_blockaded(pos):
    print("Piece at 3,3 is blockaded!")
```

is_in_bounds()

Checks if a Vector2i position is within the 16x8 board.

- **Syntax Example:**

```
GDScript
var pos = Vector2i(16, 7) # Out of bounds
if not move_validator.is_in_bounds(pos):
    print("Position is invalid.")
```

is_in_enemy_territory()

Checks if a pos is on the opponent's side of the board for the given color. (White's enemy territory is $x \geq 8$; Black's is $x \leq 7$).

- **Syntax Example:**

```
GDScript
var pos = Vector2i(9, 4)
if board.is_in_enemy_territory(pos, "white"):
    print("White has an invader!")
```

is_move_valid()

Checks if a *specific* move from from_pos to to_pos is valid for the piece. This is the core function for validating a standard move.

- **Syntax Example:**

```
GDScript
var piece_node = ...
var from_pos = Vector2i(0, 0)
var to_pos = Vector2i(0, 3) # A 3-step move for a Square
if move_validator.is_move_valid(piece_node, from_pos, to_pos):
    # Execute the move
    pass
```

is_path_clear_for_capture()

Checks if the intermediate squares between from_pos and to_pos are empty. Unlike is_move_valid, this function *allows* the to_pos to be occupied (as it would be during a capture).

- **Syntax Example:**

```
GDScript
var piece_node = ...
var from_pos = Vector2i(0, 0)
var to_pos = Vector2i(0, 3) # Target is at (0, 3)
if move_validator.is_path_clear_for_capture(piece_node, from_pos,
to_pos):
    # Now check if the values match for capture
    pass
```

is_square_empty()

Checks if a square at pos is both in bounds and has no piece on it.

- **Syntax Example:**

```
GDScript
var pos = Vector2i(5, 5)
if move_validator.is_square_empty(pos):
    print("Square 5,5 is empty.")
```

`is_within_capture_range()`

Checks if a `to_pos` is reachable by a piece from `from_pos` for a capture. This means the distance matches the piece's move (1, 2, or 3) and the path is clear (destination can be occupied).

- **Syntax Example:**

```
GDScript
# Check if a Triangle at (1,1) can target a piece at (1,3)
var piece_node = ... # A Triangle piece
var from_pos = Vector2i(1, 1)
var to_pos = Vector2i(1, 3)
if move_validator.is_within_capture_range(piece_node, from_pos, to_pos):
    print("Target is in range.")
```

`is_within_one_move()`

Checks if a `to_pos` is *exactly* one valid move away. This is used for "Equality" (Number) captures. It is essentially an alias for `is_move_valid` but is named for clarity in capture logic.

- **Syntax Example:**

```
GDScript
# Note: This function's name is misleading in the code. It's not a generic
# "one move" check, but checks if the to_pos is *empty* and reachable.
# The 'is_within_capture_range' is more appropriate for captures.
var piece_node = ...
var from_pos = Vector2i(1, 1)
var to_pos = Vector2i(1, 3)
if move_validator.is_within_one_move(piece_node, from_pos, to_pos):
    print("Can move to (1,3).")
```

6. AI & Strategic Analysis

These functions are used to evaluate the board, score positions, and select moves for the AI player ("Professor Pyramid"). Most are in **board.gd**.

`calculate_center_of_gravity()`

Calculates the average position (as a `Vector2(avg_row, avg_col)`) of all pieces for a given `piece_color`. Can be filtered to only include "ranking" pieces near enemy territory and can use a simulated `board_state_override`.

- **Syntax Example:**

```
GDScript
# Get COG for all white pieces on the live board
var cog_white = board.calculate_center_of_gravity("white")

# Get COG for black's "ranking" pieces on a simulated board
var cog_black_rank = board.calculate_center_of_gravity("black", true, sim_board_state)
```

`calculate_position_score()`

Calculates the positional score for a player based on captures, value of captures, and the number of "invader pairs."

- **Syntax Example:**

```
GDScript
var white_score_data = board.calculate_position_score("white")
print("White's score: ", white_score_data.total_score)
```

`does_move_complete_invader_fireteam()`

Checks if a specific move (from `from_pos` to `to_pos`) would result in the piece landing in enemy territory and completing a 3-piece progression with two *other* friendly pieces already there.

- **Syntax Example:**

```
GDScript
var piece_node = ...
var from_pos = Vector2i(7, 0)
var to_pos = Vector2i(10, 0) # Moving into enemy territory
var check = board.does_move_complete_invader_fireteam(piece_node, from_pos, to_pos,
"white")
if check.completes:
    print("This move completes a fireteam!")
```

`execute_ai_turn()`

The main function for the AI. It calls `select_best_move()`, executes the chosen move, and automatically advances the turn.

- **Syntax Example:**

```
GDScript
# Called automatically by 'advance_to_next_player' if it's Black's turn
board.execute_ai_turn()
```

`get_all_legal_moves_prioritized()`

Returns all legal moves for a color, but sorts them so that moves completing a "fireteam" (invader progression) are at the top of the list. Can operate on the live board or a simulated `board_state`.

- **Syntax Example:**

```
GDScript
var moves = board.get_all_legal_moves_prioritized("white", sim_board_state)
var best_move = moves[0] # This is either a fireteam move or a regular move
```

`get_all_possible_captures_from_state()`

An AI-facing function in `move_validator.gd`. It calculates all captures for a *single piece* based *only* on a `simulated board_state` array, rather than live `Node2D` objects.

- **Syntax Example:**

```
GDScript
var piece_id = "T072_1"
var pos = Vector2i(2, 1)
var captures = move_validator.get_all_possible_captures_from_state(piece_id, pos, sim_board_state)
```

`get_best_move_suggestion()`

A wrapper function that gets the prioritized list of moves and returns the best one (the first in the list). Used by the "Hint" button.

- **Syntax Example:**

```
GDScript
var hint_move = board.get_best_move_suggestion("white")
print("Hint: Move from ", hint_move.start_pos, " to ", hint_move.end_pos)
```

`get_valid_moves_from_state()`

An AI-facing function in `move_validator.gd`. It calculates all valid *destinations* for a piece based *only* on a `board_state` array.

- **Syntax Example:**

```
GDScript
var piece_id = "T072_1"
var pos = Vector2i(2, 1)
var destinations = move_validator.get_valid_moves_from_state(piece_id, pos,
sim_board_state)
```

`is_an_advance()`

Checks if a move (from `start_pos` to `end_pos`) would cause the player's "center of gravity" to move closer to the enemy's side of the board. Used as an AI tie-breaker.

- **Syntax Example:**

```
GDScript
var start_pos = Vector2i(1, 1)
var end_pos = Vector2i(4, 1)
if board.is_an_advance(start_pos, end_pos, "white"):
    print("This is an advancing move.")
```

`select_best_move()`

The core AI brain. It iterates through all legal moves, simulates each one (including pre- and post-move captures), scores the resulting board state, and selects the move with the highest score. Uses `is_an_advance()` as a tie-breaker.

- **Syntax Example:**

```
GDScript
var best_move_array = board.select_best_move("black") # [Vector2i, Vector2i]
var from_pos = best_move_array[0]
var to_pos = best_move_array[1]
```

`update_all_scores()`

Recalculates the positional scores for both White and Black and updates their respective UI labels.

- **Syntax Example:**

```
GDScript
# Called after captures or at the end of a turn
board.update_all_scores()
```

7. Game Log & Utility

These functions, found in **board.gd**, are used for game logging, exporting, and coordinate conversions.

`coordinate_to_algebraic_gd()`

Converts a (col, row) coordinate (e.g., (0, 0)) to Rithmomachia's algebraic notation (e.g., "a1").

- **Syntax Example:**

```
GDScript
var alg_notation = board.coordinate_to_algebraic_gd(15, 7)
print(alg_notation) # Output: "p8"
```

`export_game_log_to_text()`

Generates a human-readable string summarizing the entire game, including all turns, moves, captures, and the final victory condition.

- **Syntax Example:**

```
GDScript
var log_string = board.export_game_log_to_text()
file.store_string(log_string)
```

`finalize_turn_log()`

Saves the `current_turn_log` dictionary into the main `game_log` array and updates progression tracking.

- **Syntax Example:**

```
GDScript
# Called by 'advance_to_next_player'
board.finalize_turn_log()
```

log_capture()

Adds a detailed capture entry to the current_turn_log dictionary.

- **Syntax Example:**

```
GDScript
board.log_capture(
    attacker_piece_node,
    attacker_pos,
    victim_piece_node,
    victim_pos,
    "equality",
    captured_value,
    false,
    null
)
```

log_move()

Adds the move details to the current_turn_log dictionary.

- **Syntax Example:**

```
GDScript
board.log_move(piece_node, from_pos, to_pos)
```